

documentación

podemos ver una base de datos my sql con sus respectivas relaciones entre ellas, pero hay que tener en cuenta que probablemente esta no sea definitiva.

La base de datos que usaremos como ya lo dije será mysql que es una base de datos relacional para que de esta manera se me haga más fácil tener una estructura correcta y relacional entre las tablas de la base de datos.

Tablas de la base de datos.

Las tablas descritas en la base de datos son los modelos ya plasmados en código los cuales son, appointment, patient, secretary, staff, therapist, special_therapist.

Relaciones claves entre la base de datos

Relaciones clave:

- **appointments.patient_id → patients.id (FK)**
- **appointments.therapist_id → therapists.id (FK)**
- **appointments.room_id → rooms.id (FK)**
- **therapist_specialties.therapist_id → therapists.id (FK)**

hay que tener en cuenta que estas se relaciones tomando en cuenta los primary key de las tablas, para de esta tener relaciones de uno a mucho de muchos a muchos, etc.

Ejemplo de implementación

Ahora les mostrare un ejemplo para que puedan entender en un contexto mas sencillo y no tan complejo como el proyecto que llevamos acabo.

1. Preparar el entorno

- Crear un proyecto en Spring Initializr: Vaya a start.spring.io, seleccione las dependencias necesarias como Spring Web, Spring Data JPA y el controlador de su base de datos (por ejemplo, MySQL Driver o H2).
- Configurar la base de datos en application.properties: Agregue las propiedades de conexión. Para una base de datos H2 en memoria, podría ser:

properties

spring.datasource.url=jdbc:h2:mem:testdb

spring.datasource.driver-class-name=org.h2.Driver

spring.datasource.username=sa

spring.datasource.password=password

spring.jpa.database-platform=org.hibernate.dialect.H2Dialect

spring.jpa.hibernate.ddl-auto=update

2. Crear la entidad

- Cree una clase de modelo Java (por ejemplo, Chiste) y añada la anotación `@Entity`. Esta clase representará una tabla en la base de datos.

java

```
import jakarta.persistence.Entity;  
  
import jakarta.persistence.GeneratedValue;  
  
import jakarta.persistence.GenerationType;  
  
import jakarta.persistence.Id;
```

`@Entity`

```
public class Chiste {  
  
    @Id  
  
    @GeneratedValue(strategy = GenerationType.IDENTITY)  
  
    private int id;  
  
    private String texto;  
  
    private String autor;  
  
    // Constructores, getters y setters  
  
}
```

3. Crear el repositorio

- Cree una interfaz que extienda `JpaRepository` o `CrudRepository`. Spring Data JPA generará automáticamente la implementación para realizar operaciones CRUD.

java

```
import org.springframework.data.jpa.repository.JpaRepository;  
import org.springframework.stereotype.Repository;
```

`@Repository`

```
public interface ChisteRepository extends  
JpaRepository<Chiste, Integer> {
```

```
    // Aquí se pueden definir métodos de consulta personalizados  
}
```

4. Usar el repositorio en un servicio o controlador

- Inyecte la interfaz del repositorio en una clase de servicio o controlador para interactuar con la base de datos.

java

```
import  
org.springframework.beans.factory.annotation.Autowired;  
import org.springframework.web.bind.annotation.*;
```

`@RestController`

`@RequestMapping("/chistes")`

```
public class ChisteController {  
  
    @Autowired  
    private ChisteRepository chisteRepository;  
  
    @GetMapping  
    public java.util.List<Chiste> obtenerTodosLosChistes() {  
        return chisteRepository.findAll();  
    }  
  
    @PostMapping  
    public Chiste crearChiste(@RequestBody Chiste chiste) {  
        return chisteRepository.save(chiste);  
    }  
}
```

Podemos ver este ejemplo sacado de internet para que entiendan como se usaría o como se conectaría la base de datos desde el backend.

Les dejare unas imágenes para que puedan ver el código de la tabla en mysql y puedan entender las relaciones entre ellas y sus primary key.